HIP课程作业

Lab1

Exercise: Thread Index Calculation

Fill in the global index for each thread:

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	100
2	128	50	306
5	64	0	320

Exercise 1: GPU Specifications

Based on the output above, record your GPU's specifications:

Property	Your GPU Value	Significance
Device Name	AMD RYZEN AI MAX+ PRO 395 w/ Radeon 8060S	GPU model identifier
Compute Units	32	Number of parallel execution units
Wavefront Size	Radeon 8060S Graphics	Threads per warp (**32 on RDNA**, 64 on CDNA)
Max Threads per Block	40	Upper limit for blockDim
Max Shared Memory per Block	32(0x20)	LDS available per block

Question: How many total threads can your GPU execute simultaneously if all CUs are fully utilized?

Your calculation: 1280

Kernel Structure

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {
    // Step 1: Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x; // Fill in

    // Step 2: Boundary check (important when N is not divisible by block size)
    if (idx < N) {  // Fill in the condition
        // Step 3: Perform computation
         C[idx] = A[idx] + B[idx]; // Fill in
    }
}
```

Exercise 2: Implement the Kernel

Complete the kernel in `vector_add_kernel.hip` based on the structure above.

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

Exercise 3: Launch Configuration Calculation

Given `N = 1000` elements, complete the table:

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	ceil(1000/64)	16	1024	24
128	ceil(1000/128)	8	1024	24
256	ceil(1000/256)	4	1024	24
512	ceil(1000/512)	2	1024	24

Question: Which block size gives the best thread utilization? Why might this not always be the best choice?

Your answer:
这几个 block size 的线程利用率相同，都是 1000 / 1024 = 97.66%。但线程利用率最高不一定代表性能最好，因为实际性能还会受到 occupancy、wavefront 对齐、寄存器使用量、共享内存使用量以及调度开销等因素影响。

Exercise 4: Analyze Your Results

Based on the multi-trial results above:

1. Which block size achieved the **lowest mean execution time**?
   64

2. Which block size achieved the **highest memory bandwidth**?
   64
3. Are the differences between block sizes **statistically significant** (look at error bars)?
   Not clearly among the fastest wave-aligned block sizes. BS=64, 128, and 1024 have close mean times, while several configurations show large standard deviations and outliers. BS=16 is generally worse.
4. Why does Vector Add performance depend more on bandwidth than on block size?

Your answer:
在我的实验结果中，block size 为 64 时平均执行时间最低，约为 0.0182 ms，因此它也对应最高的内存带宽。不同 block size 之间的差异不一定都具有明显统计意义。BS=64、128、1024 的平均时间比较接近，而一些配置存在较大的标准差和异常值。BS=16 通常较慢，因为它小于 wavefront size。Vector Add 的性能更依赖内存带宽，而不是 block size，因为每个元素只进行一次加法运算，但需要从全局内存读取 A 和 B，并写回 C。计算量很小，访存开销占主导，所以它是 memory-bandwidth bound。

Exercise 5: Wavefront Analysis

Question 1: A kernel is launched with block size 100. How many wavefronts does each block require? How many lanes are wasted?

- Wavefronts per block: 4 (hint: $\lceil 100/32 \rceil$)
- Wasted lanes: 28 (hint: wavefronts × 32 − 100)

Question 2: If the same kernel were compiled with `-mwavefrontsize64`, how would the answers change?

- Wavefronts per block (wave64): 2 (hint: $\lceil 100/64 \rceil$)
- Wasted lanes (wave64): 28

Question 3: What happens when threads within a wavefront take different branches (if-else)?

Your answer: 当同一个 wavefront 内的线程进入不同的 if-else 分支时，会发生分支发散。GPU 会串行执行不同分支，并屏蔽当前分支中不活跃的线程，因此有效并行度会降低，性能可能下降。

Exercise 6: Occupancy Calculation

For Radeon 8060S (RDNA 3.5): 20 CUs, Warp Size = 32, Max Blocks per CU = 64, Max Threads per Block = 1024

To calculate occupancy, you need to know the max wavefronts per CU (check `rocminfo` output).

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	limited by max blocks (64)	64	每个 block 只有 1 个 wavefront，受最大 block 数限制
64	2	32	64	受 max wavefronts per CU 限制
128	4	16	64	受 max wavefronts per CU 限制
256	8	8	64	受 max wavefronts per CU 限制
512	16	4	64	受 max wavefronts per CU 限制

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
1024	32	2	64	每个 block 很大，每个 CU 只能同时放 2 个 block

Note: 根据 rocminfo 输出，我的 GPU 是 Radeon 8060S Graphics，具有 40 个 Compute Units，Wavefront Size 为 32。对于每个 CU，active wavefronts 的上限为 64。Block size 越大，每个 block 包含的 wavefronts 越多，因此每个 CU 能同时容纳的 block 数越少。虽然 block size 从 32 到 1024 时 blocks that fit 不同，但这些配置都可以达到 64 个 active wavefronts，因此理论上都能达到满 occupancy。

Exercise 7: Analyze Sub-Wavefront Results

Based on the multi-trial results above:

- Mean time for block size 8 (sub-wave): 0.0539 ms
- Mean time for block size 16 (sub-wave): 0.0345 ms
- Mean time for block size 32 (wave-aligned): 0.0412 ms
- Mean time for block size 256 (large): 0.0315 ms

Question 1: Is there a noticeable performance penalty for sub-wavefront block sizes (8, 16) compared to wave-aligned sizes (32, 64, 128)?

Your answer: 有明显的性能损失。Block size 8 和 16 都小于 wavefront size 32，属于 sub-wavefront block size，不能充分利用一个 wavefront 中的所有 lane。从实验结果看，BS=8 的平均时间约为 0.0539 ms，BS=16 的平均时间约为 0.0345 ms，而 wave-aligned 的 BS=64 只有约 0.0181 ms，BS=128 约为 0.0198 ms。因此 sub-wavefront block sizes 通常比 wave-aligned block sizes 更慢。

Question 2: On this RDNA 3.5 GPU, the wavefront size is 32. If you were on an MI100 (wavefront = 64), which block sizes would become sub-wavefront?

Your answer: 在 RDNA 3.5 上，wavefront size 是 32，所以小于 32 的 block size，比如 8 和 16，是 sub-wavefront。如果换成 MI100，wavefront size 是 64，那么所有小于 64 的 block size 都会变成 sub-wavefront。因此在这组实验中，8、16、32 都会变成 sub-wavefront block size。

13. Challenge Exercise

Modify the kernel to process **multiple elements per thread**. This is a common technique when N is very large.

```
__global__ void vector_add_multi(const float* A, const float* B, float* C,
                                 int N, int elements_per_thread) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;  // Fill in: total number of threads

    for (int i = tid; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}
```

Question: What is the advantage of this approach over the 1:1 thread-to-element mapping?

1. 减少 kernel launch 开销：当 N 非常大时，不需要启动大量的 block 和 thread，可以用较少的线程通过循环处理更多数据，减少调度开销。
2. 更好的内存访问局部性：同一个线程连续访问的元素在内存中可能相邻，有利于缓存命中。
3. 更灵活的线程配置：不受 N 的大小限制，可以用固定的 block 数量处理任意大小的 N。
4. 更好的计算与访存比：每个线程处理多个元素，可以更好地隐藏内存延迟，提高 ALU 利用率。

Lab2

Exercise 1: 2D Indexing

Given `blockDim = (32, 32)`, calculate the global indices:

blockIdx	threadIdx	Global (idx, idy)
(0, 0)	(5, 10)	(5, 10)
(1, 0)	(0, 0)	(32, 0)
(2, 3)	(15, 20)	(79, 116)

Question: For a matrix with `rows=100, cols=200`, how many blocks are needed?

Your calculation: 28

Exercise 2: Understand the Index Transformation

For a 4×6 matrix (rows=4, cols=6), fill in the table:

Thread (idx, idy)	Input Index	Output Index
(0, 0)	0*6+0 = 0	0*4+0 = 0
(1, 0)	0*6+1 = 1	1*4+0 = 4
(0, 1)	1*6+0 = 6	0*4+1 = 1
(3, 2)	2*6+3 = 15	3*4+2 = 14

Question: Why is the write index formula `idx * rows + idy` instead of `idx * cols + idy`?

Your answer: 输入矩阵的列数是 cols，输出矩阵（转置后）的列数是 rows，所以写索引用 rows 而不是 cols

Exercise 3: Performance Analysis

Record execution times:

Test	Dimensions	Elements	Time
1	16×16	256	0.0079
2	128×32	4,096	0.0092

Test	Dimensions	Elements	Time
3	1×1024	1,024	0.0075
4	1001×2001	2,003,001	0.1117

Question: The large test has ~500x more elements than test 2. Is the time 500x longer? Why or why not?

Your answer:不是 500x 的时间关系。
Test 2 有 4,096 个元素，Test 4 有 2,003,001 个元素，元素数量比约为 489x
但时间从 0.0092 ms 增加到 0.1117 ms，只增加了约 12x
原因：

1. Kernel Launch Overhead 主导小矩阵：Test 1-3 的元素数量很少，kernel 执行时间非常短（< 0.01 ms），大部分时间实际上是 kernel launch overhead，而不是真正的 GPU 计算时间。
2. 大矩阵展现了真实的内存带宽：Test 4 有足够多的元素，kernel launch overhead 被摊薄，真实的内存访问模式开始主导性能。但由于写操作是 strided（非合并），内存带宽利用率只有约 143 GB/s，远低于 256 GB/s 的峰值。
3. Per-element cost 差异巨大：
   - Test 1: ~30.9 ns/element
   - Test 2: ~2.2 ns/element
   - Test 3: ~7.3 ns/element
   - Test 4: ~0.06 ns/element
     大矩阵的 per-element cost 反而更低，因为 launch overhead 被更多元素分摊。

Exercise 4: Coalescing Analysis

For a 1024×1024 matrix with 32×32 thread blocks (one wave32 = one row of the block):

Question 1: When reading `input[idy * cols + idx]`, are consecutive threads (in `threadIdx.x`) reading consecutive addresses?

Your answer: 是的，是coalesced

- 连续线程（threadIdx.x = 0, 1, 2, ...）读取的地址是：idy * cols + 0, idy * cols + 1, idy * cols + 2, ...
- 这些地址在内存中是连续的，因此 GPU 可以将它们合并为一次内存事务（128 bytes = 32 floats）。

Question 2: When writing `output[idx * rows + idy]`, what is the stride between addresses written by consecutive threads?

Your answer: stride = rows（即 1024 个 float = 4096 bytes）

- 连续线程（threadIdx.x = 0, 1, 2, ...）写入的地址是：0 * rows + idy, 1 * rows + idy, 2 * rows + idy, ...
- 相邻线程写入的地址间隔 = rows = 1024 个 float
- 在字节单位上，stride = 1024 × 4 bytes = 4096 bytes

Question 3: If each memory transaction fetches 128 bytes (32 floats), how many transactions are needed for 32 threads (one wavefront) to write non-coalesced?

Your answer: 需要 32 次内存事务

- 每个线程写入的地址间隔是 4096 bytes
- 每次内存事务只能获取 128 bytes（32 floats）
- 由于 32 个线程写入的地址之间间隔 4096 bytes，它们完全不在同一个 128-byte cache line 内
- 因此每个线程都需要一次独立的内存事务
- 总共需要 32 次内存事务（而不是合并访问时的 1 次）

Exercise 5: Tiled Transpose Analysis

Question 1: Why do we add `+1` to the shared memory declaration (`tile[BLOCK_SIZE][BLOCK_SIZE + 1]`)?

Your answer: 避免 Bank Conflict

- AMD GPU 的 shared memory（LDS）被分成多个 bank（通常是 32 个 bank）
- 当同一个 wavefront 中的多个线程同时访问同一个 bank 的不同地址时，会发生 bank conflict，导致串行化访问
- 在转置操作中，线程从 tile[threadIdx.y][threadIdx.x] 读取，然后写入 tile[threadIdx.x][threadIdx.y]
- 如果不加 padding，转置后同一列的线程会访问同一个 bank
- +1 padding 使得相邻列的数据落在不同的 bank 上，从而避免 bank conflict

Question 2: In the write step, why do we swap blockIdx.x and blockIdx.y?

Your answer: 因为转置后块的位置需要交换

- 输入矩阵中，block 位于 (blockIdx.x, blockIdx.y) 负责处理输入的 (x, y) 区域
- 转置后，这个 block 的数据应该写到输出的 (y, x) 区域
- 因此输出位置计算为：
  - x = blockIdx.y * BLOCK_SIZE + threadIdx.x
  - y = blockIdx.x * BLOCK_SIZE + threadIdx.y
- 这样每个 block 负责写入转置后的对应位置

Question 3: How much shared memory does each block use for BLOCK_SIZE=32?

Your calculation: 4224 bytes

12. Challenge Exercises

Challenge 1: Implement Tiled Transpose

Modify `main.hip` to use the shared memory tiling approach from Section 9. Compare performance with the naive version.

```cpp
// Tiled transpose kernel using shared memory
__global__ void matrix_transpose_tiled(const float* input, float* output, int rows, int cols) {
    __shared__ float tile[BLOCK_SIZE][BLOCK_SIZE + 1];  // +1 to avoid bank conflicts

    int x = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    int y = blockIdx.y * BLOCK_SIZE + threadIdx.y;

    // Step 1: Read coalesced from global memory into shared memory
    if (x < cols && y < rows) {
        tile[threadIdx.y][threadIdx.x] = input[y * cols + x];
    }
    __syncthreads();

    // Step 2: Calculate transposed output position
    x = blockIdx.y * BLOCK_SIZE + threadIdx.x;
    y = blockIdx.x * BLOCK_SIZE + threadIdx.y;

    // Step 3: Write coalesced from shared memory to global memory
    if (x < rows && y < cols) {
        output[y * rows + x] = tile[threadIdx.x][threadIdx.y];
    }
}
```

```
=== Performance Comparison ===
Matrix: 16x16 (256 elements)
NAIVE  - Time: 0.00795495 ms, BW: 0.25745 GB/s
TILED  - Time: 0.0071875 ms, BW: 0.284939 GB/s
SPEEDUP: 1.10678x
KERNEL_MS_NAIVE 0.00795495
KERNEL_MS_TILED 0.0071875
=== Performance Comparison ===
Matrix: 128x32 (4096 elements)
NAIVE  - Time: 0.00918125 ms, BW: 3.56901 GB/s
TILED  - Time: 0.0074039 ms, BW: 4.42578 GB/s
SPEEDUP: 1.24006x
KERNEL_MS_NAIVE 0.00918125
KERNEL_MS_TILED 0.0074039
=== Performance Comparison ===
Matrix: 1x1024 (1024 elements)
NAIVE  - Time: 0.00718945 ms, BW: 1.13945 GB/s
TILED  - Time: 0.00791285 ms, BW: 1.03528 GB/s
SPEEDUP: 0.908579x
KERNEL_MS_NAIVE 0.00718945
KERNEL_MS_TILED 0.00791285
=== Performance Comparison ===
Matrix: 1001x2001 (2003001 elements)
NAIVE  - Time: 0.111907 ms, BW: 143.19 GB/s
TILED  - Time: 0.0281567 ms, BW: 569.101 GB/s
SPEEDUP: 3.97445x
KERNEL_MS_NAIVE 0.111907
KERNEL_MS_TILED 0.0281567
```

1. **大矩阵加速显著**：1001×2001 矩阵上 TILED 版本加速了 **3.97x**，带宽从 143 GB/s 提升到 **569 GB/s**，远超 LPDDR5x 峰值 256 GB/s。这说明大矩阵的数据很可能被 L2 cache 命中，tiled 版本因为访问模式更规则，cache 命中率更高。

2. **小矩阵无优势甚至更慢**：1×1024 矩阵上 TILED 反而慢了 9%（0.91x）。原因是：
   - 元素太少，shared memory 的 `__syncthreads()` 同步开销和 extra load/store 开销超过了 coalescing 带来的收益
   - 1×1024 本质上是一维数据，转置后变成 1024×1，tiled 的 2D block 策略没有优势
3. **Kernel Launch Overhead 主导**：16×16 和 128×32 的绝对时间差异很小（< 0.002 ms），说明 kernel launch 固定开销占主导，真正的内存访问优化被掩盖。
4. **Tiled 的核心优势**：将 non-coalesced 的全局内存写入转化为 coalesced 写入，在大数据量时效果显著。

Challenge 2: Bank Conflict Analysis

Test with and without the `+1` padding in shared memory. Measure the performance difference.

```
__global__ void matrix_transpose_tiled_nopad(const float* input, float* output, int rows, int cols) {
    __shared__ float tile[BLOCK_SIZE][BLOCK_SIZE];  // NO padding - may cause bank conflicts

    int x = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    int y = blockIdx.y * BLOCK_SIZE + threadIdx.y;

    // Step 1: Read coalesced from global memory into shared memory
    if (x < cols && y < rows) {
        tile[threadIdx.y][threadIdx.x] = input[y * cols + x];
    }
    __syncthreads();

    // Step 2: Calculate transposed output position
    x = blockIdx.y * BLOCK_SIZE + threadIdx.x;
    y = blockIdx.x * BLOCK_SIZE + threadIdx.y;

    // Step 3: Write from shared memory to global memory
    if (x < rows && y < cols) {
        output[y * rows + x] = tile[threadIdx.x][threadIdx.y];
    }
}
```

```
=== Bank Conflict Analysis ===
Matrix: 16x16 (256 elements)
NAIVE      - Time: 0.007983 ms, BW: 0.256545 GB/s
TILED+PAD  - Time: 0.0071895 ms, BW: 0.28486 GB/s (speedup: 1.11037x)
TILED-NOPAD- Time: 0.0081132 ms, BW: 0.252428 GB/s (speedup: 0.983952x)
PAD vs NOPAD: 1.12848x
KERNEL_MS_NAIVE 0.007983
KERNEL_MS_TILED_PAD 0.0071895
KERNEL_MS_TILED_NOPAD 0.0081132
=== Bank Conflict Analysis ===
Matrix: 128x32 (4096 elements)
NAIVE      - Time: 0.00921735 ms, BW: 3.55503 GB/s
TILED+PAD  - Time: 0.0073879 ms, BW: 4.43536 GB/s (speedup: 1.24763x)
TILED-NOPAD- Time: 0.0092113 ms, BW: 3.55737 GB/s (speedup: 1.00066x)
PAD vs NOPAD: 1.24681x
KERNEL_MS_NAIVE 0.00921735
KERNEL_MS_TILED_PAD 0.0073879
KERNEL_MS_TILED_NOPAD 0.0092113
=== Bank Conflict Analysis ===
Matrix: 1x1024 (1024 elements)
NAIVE      - Time: 0.0074881 ms, BW: 1.094 GB/s
TILED+PAD  - Time: 0.00791085 ms, BW: 1.03554 GB/s (speedup: 0.946561x)
TILED-NOPAD- Time: 0.0073057 ms, BW: 1.12132 GB/s (speedup: 1.02497x)
PAD vs NOPAD: 0.923504x
KERNEL_MS_NAIVE 0.0074881
KERNEL_MS_TILED_PAD 0.00791085
KERNEL_MS_TILED_NOPAD 0.0073057
=== Bank Conflict Analysis ===
Matrix: 1001x2001 (2003001 elements)
NAIVE      - Time: 0.111841 ms, BW: 143.275 GB/s
TILED+PAD  - Time: 0.028331 ms, BW: 565.6 GB/s (speedup: 3.94766x)
TILED-NOPAD- Time: 0.0437318 ms, BW: 366.415 GB/s (speedup: 2.55743x)
PAD vs NOPAD: 1.5436x
KERNEL_MS_NAIVE 0.111841
KERNEL_MS_TILED_PAD 0.028331
KERNEL_MS_TILED_NOPAD 0.0437318
```

1. **Bank Conflict 在大矩阵上影响显著**：1001×2001 矩阵上，无 padding 版本比有 padding 慢了 **1.54x**（0.0437 ms vs 0.0283 ms），带宽从 566 GB/s 降到 366 GB/s，损失了约 **35%** 的性能。
2. **Bank Conflict 的原理**：
   - RDNA 3.5 的 LDS（shared memory）有 32 个 bank
   - `tile[32][32]` 中，同一列的元素（`tile[0][k]`，`tile[1][k]`，...，`tile[31][k]`）落在同一个 bank
   - 转置读取时，线程访问 `tile[threadIdx.x][threadIdx.y]`，同一行的线程访问不同列的同一行位置，导致多个线程访问同一个 bank 的不同地址 → **bank conflict**
   - `tile[32][33]` 的 +1 padding 使得每列的起始地址错开一个 bank，消除了 conflict
3. **小矩阵上结果不稳定**：1×1024 上 NOPAD 反而更快（0.92x），这是因为数据量太小，noise 和 cache 效应主导，bank conflict 的影响被掩盖。
4. **结论**：`+1` padding 是一个重要的优化手段，在大矩阵上可以带来 **30-50%** 的性能提升，应该始终使用。

Challenge 3: In-Place Transpose

For square matrices, implement in-place transpose using only O(1) extra memory.

```
__global__ void matrix_transpose_inplace(float* matrix, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int idy = blockIdx.y * blockDim.y + threadIdx.y;

    // Only process upper triangle (i < j) to avoid double swap
    if (idx < idy && idx < N && idy < N) {
        // Swap matrix[idy][idx] and matrix[idx][idy]
        float temp = matrix[idy * N + idx];
        matrix[idy * N + idx] = matrix[idx * N + idy];
        matrix[idx * N + idy] = temp;
    }
}
```

```
=== In-Place Transpose Analysis (Square Matrix) ===
Matrix: 16x16
OUT-OF-PLACE - Time: 0.00795295 ms, BW: 0.257515 GB/s
KERNEL_MS_OUT_OF_PLACE 0.00795295
IN-PLACE     - Time: 0.00237045 ms, BW: 0.863971 GB/s
KERNEL_MS_IN_PLACE 0.00237045
Verification: PASS
=== In-Place Transpose Analysis (Square Matrix) ===
Matrix: 128x128
OUT-OF-PLACE - Time: 0.0096782 ms, BW: 13.543 GB/s
KERNEL_MS_OUT_OF_PLACE 0.0096782
IN-PLACE     - Time: 0.00336235 ms, BW: 38.9823 GB/s
KERNEL_MS_IN_PLACE 0.00336235
Verification: PASS
=== In-Place Transpose Analysis (Square Matrix) ===
Matrix: 1024x1024
OUT-OF-PLACE - Time: 0.313154 ms, BW: 26.7875 GB/s
KERNEL_MS_OUT_OF_PLACE 0.313154
IN-PLACE     - Time: 0.14125 ms, BW: 59.3882 GB/s
KERNEL_MS_IN_PLACE 0.14125
Verification: PASS
```

Challenge 4: Rectangular Tile Sizes

Experiment with non-square tile sizes (e.g., 32×8 or 16×64). When might these be beneficial?

```
template <int TILE_X, int TILE_Y>
__global__ void matrix_transpose_tiled_rect(const float* input, float* output, int rows, int cols) {
    __shared__ float tile[TILE_Y][TILE_X + 1];

    int x = blockIdx.x * TILE_X + threadIdx.x;
    int y = blockIdx.y * TILE_Y + threadIdx.y;

    if (x < cols && y < rows)
        tile[threadIdx.y][threadIdx.x] = input[y * cols + x];
    __syncthreads();

    x = blockIdx.y * TILE_Y + threadIdx.x;
    y = blockIdx.x * TILE_X + threadIdx.y;

    if (x < rows && y < cols)
        output[y * rows + x] = tile[threadIdx.x][threadIdx.y];
}

// Wrapper to launch with different tile sizes
template <int TILE_X, int TILE_Y>
void launch_transpose(const float* d_input, float* d_output, int rows, int cols, float& kernel_ms) {
    dim3 threadsPerBlock(TILE_X, TILE_Y);
    dim3 blocksPerGrid((cols + TILE_X - 1) / TILE_X,
                       (rows + TILE_Y - 1) / TILE_Y);

    hipEvent_t ev_start, ev_stop;
    hipEventCreate(&ev_start);
    hipEventCreate(&ev_stop);

    // Warm-up
    matrix_transpose_tiled_rect<TILE_X, TILE_Y><<<blocksPerGrid, threadsPerBlock>>>(d_input, d_output, rows, cols);
    hipDeviceSynchronize();

    const int REPEATS = 20;
    hipEventRecord(ev_start);
    for (int r = 0; r < REPEATS; ++r) {
        matrix_transpose_tiled_rect<TILE_X, TILE_Y><<<blocksPerGrid, threadsPerBlock>>>(d_input, d_output, rows, cols)
    }
    hipEventRecord(ev_stop);
    hipEventSynchronize(ev_stop);

    float total_ms = 0.0f;
    hipEventElapsedTime(&total_ms, ev_start, ev_stop);
    kernel_ms = total_ms / REPEATS;

    hipEventDestroy(ev_start);
    hipEventDestroy(ev_stop);
}
```

```
=== Rectangular Tile Analysis ===
Matrix: 128x32
32x32  - Time: 0.00742195 ms, BW: 4.41501 GB/s
32x8   - Time: 0.00732375 ms, BW: 4.47421 GB/s
8x32   - Time: 0.00386925 ms, BW: 8.46882 GB/s
16x64  - Time: 0.0024927 ms, BW: 13.1456 GB/s
64x16  - Time: 0.0024085 ms, BW: 13.6051 GB/s
=== Rectangular Tile Analysis ===
Matrix: 1x1024
32x32  - Time: 0.0078427 ms, BW: 1.04454 GB/s
32x8   - Time: 0.0073177 ms, BW: 1.11948 GB/s
8x32   - Time: 0.0077345 ms, BW: 1.05915 GB/s
16x64  - Time: 0.00920325 ms, BW: 0.89012 GB/s
64x16  - Time: 0.00237845 ms, BW: 3.44426 GB/s
=== Rectangular Tile Analysis ===
Matrix: 1001x2001
32x32  - Time: 0.0274994 ms, BW: 582.703 GB/s
32x8   - Time: 0.0234499 ms, BW: 683.329 GB/s
8x32   - Time: 0.0301885 ms, BW: 530.798 GB/s
16x64  - Time: 0.0290483 ms, BW: 551.632 GB/s
64x16  - Time: 0.0268642 ms, BW: 596.481 GB/s
```

1. **极端长宽比矩阵（128×32）**：64×16 最快（13.6 GB/s），是 32×32 的 **3x**。因为：
   - 128 行只需要 2 个 64-row block，而 32×32 需要 4 个
   - 更少的 block 意味着更少的调度开销
2. **行向量（1×1024）**：64×16 最快（3.44 GB/s），是 32×32 的 **3.3x**。因为：
   - 只有 1 行，64×16 只需要 1 个 block 覆盖所有行
   - 32×32 需要多个 block，大量线程闲置
3. **大矩阵（1001×2001）**：32×32 和 64×16 接近（582 vs 596 GB/s），差异不大。因为：
   - 数据量足够大，tile 形状的影响被摊薄
   - 内存带宽成为瓶颈，而非 block 调度

Challenge 5: Double-Buffering

Implement double-buffering to overlap memory transfers with computation for very large matrices.

```
void transpose_serial(const float* h_input, float* h_output, int rows, int cols, float& total_ms) {
    float *d_input, *d_output;
    hipMalloc(&d_input, rows * cols * sizeof(float));
    hipMalloc(&d_output, rows * cols * sizeof(float));

    hipEvent_t start, stop;
    hipEventCreate(&start);
    hipEventCreate(&stop);

    hipEventRecord(start);
    hipMemcpy(d_input, h_input, rows * cols * sizeof(float), hipMemcpyHostToDevice);

    dim3 threads(BLOCK_SIZE, BLOCK_SIZE);
    dim3 blocks((cols + BLOCK_SIZE - 1) / BLOCK_SIZE, (rows + BLOCK_SIZE - 1) / BLOCK_SIZE);
    matrix_transpose_tiled<<<blocks, threads>>>(d_input, d_output, rows, cols);
    hipDeviceSynchronize();

    hipMemcpy(h_output, d_output, rows * cols * sizeof(float), hipMemcpyDeviceToHost);
    hipEventRecord(stop);
    hipEventSynchronize(stop);
    hipEventElapsedTime(&total_ms, start, stop);

    hipEventDestroy(start);
    hipEventDestroy(stop);
    hipFree(d_input);
    hipFree(d_output);
}

// Double-buffering
void transpose_double_buffer(const float* h_input, float* h_output, int rows, int cols, float& total_ms) {
    const int chunk_rows = rows / NUM_CHUNKS;

    float *d_input[2], *d_output[2];
    hipStream_t streams[2];

    for (int i = 0; i < 2; ++i) {
        hipMalloc(&d_input[i], chunk_rows * cols * sizeof(float));
        hipMalloc(&d_output[i], chunk_rows * rows * sizeof(float));  // 转置后: chunk_rows 行 -> chunk_rows 列
        hipStreamCreate(&streams[i]);
    }

    hipEvent_t start, stop;
    hipEventCreate(&start);
    hipEventCreate(&stop);
    hipEventRecord(start);

    dim3 threads(BLOCK_SIZE, BLOCK_SIZE);
    dim3 blocks((cols + BLOCK_SIZE - 1) / BLOCK_SIZE, (chunk_rows + BLOCK_SIZE - 1) / BLOCK_SIZE);

    for (int chunk = 0; chunk < NUM_CHUNKS; ++chunk) {
        int buf = chunk % 2;
        int in_offset = chunk * chunk_rows * cols;

        // 异步 H2D: 输入的第 chunk 块 (chunk_rows × cols)
        hipMemcpyAsync(d_input[buf], h_input + in_offset,
                       chunk_rows * cols * sizeof(float),
                       hipMemcpyHostToDevice, streams[buf]);
```

```
        // 异步 Kernel: 转置 chunk_rows × cols -> cols × chunk_rows
        matrix_transpose_tiled<<<blocks, threads, 0, streams[buf]>>>(
            d_input[buf], d_output[buf], chunk_rows, cols);

        // 异步 D2H: 输出到转置后的正确位置
        // 转置后，原第 chunk 块的数据位于输出矩阵的列方向
        // 输出位置: 行 = 0..cols-1, 列 = chunk*chunk_rows .. (chunk+1)*chunk_rows-1
        for (int y = 0; y < cols; ++y) {
            int out_offset = y * rows + chunk * chunk_rows;
            int d_offset = y * chunk_rows;
            hipMemcpyAsync(h_output + out_offset, d_output[buf] + d_offset,
                           chunk_rows * sizeof(float),
                           hipMemcpyDeviceToHost, streams[buf]);
        }
    }

    hipDeviceSynchronize();
    hipEventRecord(stop);
    hipEventSynchronize(stop);
    hipEventElapsedTime(&total_ms, start, stop);

    hipEventDestroy(start);
    hipEventDestroy(stop);
    for (int i = 0; i < 2; ++i) {
        hipStreamDestroy(streams[i]);
        hipFree(d_input[i]);
        hipFree(d_output[i]);
    }
}
```

```
=== Double-Buffering Analysis ===
Matrix: 1024x1024
Serial        - Time: 5.50545 ms
Double-Buffer - Time: 15.4533 ms
Speedup: 0.356263x
Verification: PASS

=== Double-Buffering Analysis ===
Matrix: 8192x8192
Serial        - Time: 32.4311 ms
Double-Buffer - Time: 238.515 ms
Speedup: 0.135971x
Verification: PASS
```

lab3

Exercise 1: Analyze the Problem

Question 1: How many times is each input element read from global memory?

Your answer: num_bins 次

Question 2: If num_bins = 256 and N = 10M, what is the total global memory traffic?

Your calculation: N × num_bins = 10,000,000 × 256 = **2,560,000,000 reads**

Question 3: Why is this approach inefficient?

Your answer:

1. **巨大的内存流量**：每个元素被读取 num_bins 次，而不是 1 次
2. **没有利用 GPU 并行优势**：每个 block 独立工作，没有协作
3. **内存带宽成为瓶颈**：2.56B 次读取远超必要
4. **对比 Optimized 版本**：Optimized 版本每个元素只读取 1 次

Exercise 2: Understand the Optimization

Question 1: Why is atomicAdd on shared memory faster than on global memory?

Your answer: Shared memory 是片上存储，延迟低、带宽高，而且 atomic contention 只发生在同一个 block 内的线程之间，不会像 global memory 那样受到整个 grid 所有线程的竞争。

Question 2: In the merge step, how many global atomicAdd operations occur per bin?

Your answer: gridDim.x 次（即 block 数量）

Question 3: What is the purpose of the grid-stride loop pattern `for (int i = idx; i < N; i += totalthread)`？

Your answer:

1. **处理任意大小的 N**：即使 N 大于总线程数，也能覆盖所有元素
2. **保持合并访问**：相邻线程访问相邻内存地址
3. **提高占用率**：线程可以重复利用，减少 launch overhead
4. **代码简洁**：不需要复杂的边界条件处理

Exercise 3: Record Results

Fill in the execution times:

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	2.456	23.217
Naive GPU	5.606	358.986
Optimized GPU	0.185	1.778

Calculate speedups:

- Optimized vs Serial: 13.3 x(testcase2) 13.1 x(testcase4)
- Optimized vs Naive: 30.3 x(testcase2) 201.9 x(testcase4)

Exercise 4: Memory Efficiency

Question: Calculate the memory traffic reduction ratio:

$$\text{Reduction} = \frac{\text{Naive reads}}{\text{Optimized reads}} = \frac{N \times \text{num\_bins}}{N} = ?$$

Your answer: Reduction = num_bins

Exercise 5: Contention Scenarios

Scenario 1: All input values are the same (e.g., all zeros)

- What happens to performance? 性能显著下降
- Why? 所有线程都尝试对同一个 bin（bin 0）执行 atomicAdd，造成最大 contention。每个 wave32 的 32 个线程都竞争同一个内存地址，导致大量串行化。

Scenario 2: Input values are uniformly distributed across all bins

- Expected contention level: 最低（最小 contention）
- Why? 线程访问的 bin 均匀分散到所有 bins 上，每个 bin 同时被更新的概率很低。LDS 有 32 个 bank，均匀分布时多个线程同时命中不同 bank，几乎没有 conflict。

11. Challenge Exercises

Challenge 1: Implement Warp-Level Histogram

Modify the kernel to use warp-level privatization:

- Each warp maintains its own local histogram
- Reduces shared memory contention further

```cpp
__global__ void kernel_warp(const int* input, int* histogram, int N, int num_bins) {
    extern __shared__ int sdata_warp[];

    int tid = threadIdx.x;
    int warp_id = tid / WARP_SIZE;
    int lane_id = tid % WARP_SIZE;
    int warps_per_block = blockDim.x / WARP_SIZE;

    int idx = tid + blockIdx.x * blockDim.x;
    int totalthread = blockDim.x * gridDim.x;

    // Initialize all warp histograms
    #pragma unroll
    for(int i = tid; i < warps_per_block * num_bins; i += blockDim.x){
        sdata_warp[i] = 0;
    }
    __syncthreads();

    // Each warp has its own histogram
    int* warp_hist = &sdata_warp[warp_id * num_bins];

    // Accumulate to warp-level histogram
    for(int i = idx; i < N; i += totalthread){
        atomicAdd(&warp_hist[input[i]], 1);
    }
    __syncthreads();

    // Merge warp-level to global (first warp does this)
    if (warp_id == 0) {
        for(int bin = lane_id; bin < num_bins; bin += WARP_SIZE){
            int sum = 0;
            for (int w = 0; w < warps_per_block; w++) {
                sum += sdata_warp[w * num_bins + bin];
            }
            atomicAdd(&histogram[bin], sum);
        }
    }
}
```

- 实验结果

版本	时间 (ms)	加速比
Block-Level	1.694	1.0x (baseline)
Warp-Level	6.088	**0.28x**（更慢）

- 为什么 Warp-Level 更慢?

1. **Shared Memory 爆炸**：512 bins × 8 warps = 16KB，是 block-level（2KB）的 8 倍
2. **Occupancy 下降**：16KB/block 导致每个 CU 只能放更少的 block
3. **均匀分布**：contention 本来就低，privatization 没有优势，只有额外开销

- 结论

Warp-Level Privatization 适用于 **高 contention、少 bins** 的场景。在均匀分布、多 bins 的场景下反而有害。优化需要根据实际数据分布选择策略。

Challenge 2: Benchmark with Different Distributions

Modify `geninput.py` to generate:

1. Uniform distribution
2. Gaussian distribution (most values near center)
3. Highly skewed (90% of values in one bin)

Compare performance for each distribution.

```
def generate_testcase_uniform():
    N, num_bins = 10_000_000, 64
    data = [random.randint(0, num_bins - 1) for _ in range(N)]
    with open('testcases_dist/uniform.in', 'w') as f:
        f.write(f"{N} {num_bins}\n")
        f.write(" ".join(map(str, data)) + "\n")

def generate_testcase_gaussian():
    N, num_bins = 10_000_000, 64
    import math
    data = []
    for _ in range(N):
        # Box-Muller 生成高斯分布
        u1 = random.random()
        u2 = random.random()
        z = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
        val = int(num_bins / 2 + z * num_bins / 8)
        val = max(0, min(num_bins - 1, val))
        data.append(val)
    with open('testcases_dist/gaussian.in', 'w') as f:
        f.write(f"{N} {num_bins}\n")
        f.write(" ".join(map(str, data)) + "\n")

def generate_testcase_skewed():
    N, num_bins = 10_000_000, 64
    data = []
    for _ in range(N):
        if random.random() < 0.9:
            data.append(0)  # 90% 在 bin 0
        else:
            data.append(random.randint(1, num_bins - 1))
    with open('testcases_dist/skewed.in', 'w') as f:
        f.write(f"{N} {num_bins}\n")
        f.write(" ".join(map(str, data)) + "\n")
```

- 实验结果

分布	KERNEL_MS (ms)	相对性能
Uniform	0.1856	最快 (baseline)
Gaussian	0.1861	+0.3%（几乎相同）
Skewed (90% bin 0)	0.2447	**+31.8%（明显更慢）**

- 关键发现

1. **Uniform 和 Gaussian 性能接近**：64 bins 足以分散 Gaussian 分布的 contention

2. **Skewed 分布显著变慢**：90% 命中 bin 0，严重 LDS contention
3. **Privatization 的保护作用**：只慢 32% 而非几十倍，因为 contention 被限制在 block 内部

Challenge 3: Vary Number of Bins

Test performance with num_bins = {16, 64, 256, 1024}

- How does bin count affect shared memory usage?
- At what point does the optimization become less effective?

```
def generate_testcase_bins():
    N = 10_000_000
    for num_bins in [16, 64, 256, 1024]:
        array_data = [random.randint(0, num_bins - 1) for _ in range(N)]
        filename = f'testcases_dist/bins_{num_bins}.in'
        write_histogram_to_file(filename, N, num_bins, array_data)
        print(f"  {filename} generated")
```

- 实验结果

num_bins	KERNEL_MS (ms)	Shared Memory (per block)
16	0.1860	64 bytes
64	0.1857	256 bytes
256	0.1857	1 KB
1024	0.1860	4 KB

- 关键发现

1. **性能几乎不变**：从 16 bins 到 1024 bins，kernel 时间只变化了 0.2%
2. **内存带宽主导**：瓶颈是 N 次 global memory 读取，不是 shared memory 大小
3. **1024 bins 以下都是安全的**：occupancy 没有受到显著影响

- 问题回答
  - 每 block 需要 `num_bins × 4` bytes。16→64B，64→256B，256→1KB，1024→4KB
  - 当 num_bins > 4096 时，LDS 占用可能限制 occupancy。本次测试范围内性能稳定

lab4

Exercise 1: Understand Tree Reduction

Question 1: For an array of 1024 elements with block size 256, how many reduction steps occur within each block?

Your answer: 8

Question 2: After block-level reduction, how many partial sums remain?

Your answer: 4

Question 3: Why do we use `__syncthreads()` after each reduction step?

Your answer: 因为每一步 reduction 都依赖上一步所有线程写入 shared memory 的结果。使用 __syncthreads() 可以确保同一个 block 内的所有线程都完成当前步骤并把数据写入 shared memory 后，下一步 reduction 才开始。否则有些线程可能会读到尚未更新的数据，导致结果错误。

Exercise 2: Analyze Occupancy Results

From the test output, fill in the table:

Block Size	Warps/Block	Occupancy (%)	Execution Time (ms)
64	2	100.0	0.0227
128	4	100.0	0.0238
256	8	100.0	0.0250
512	16	100.0	0.0269
1024	32	100.0	0.0293

Question: Does higher occupancy always mean better performance?

Your answer:

不会。更高的 occupancy 并不总是意味着更好的性能。在这个实验中，所有 block size 的 occupancy 都是 100.0%，但执行时间并不相同。Block size 为 64 时最快，执行时间为 0.0227 ms；而 block size 为 1024 时更慢，执行时间为 0.0293 ms。这说明性能不仅取决于 occupancy，还会受到同步开销、shared memory 访问、warp efficiency、内存访问模式以及 atomic 操作数量等因素影响。对于 reduction 来说，算法本身的效率往往比单纯提高 occupancy 更重要。

Exercise 3: Think About Trade-offs

Question 1: With block size 64, how many final atomic operations occur for N = 1M elements?

Calculation: grid_size = ceil(1,000,000 / 64) = 15,625

Question 2: With block size 1024, how many final atomic operations occur?

Calculation: grid_size = ceil(1,000,000 / 1024) = 977

Question 3: Why might fewer atomic operations improve performance?

Your answer:

更少的 atomic 操作可能提高性能，因为 atomicAdd 会对同一个全局内存地址进行竞争访问。当很多 block 同时向同一个 output 累加结果时，会产生 atomic contention，使这些操作被部分串行化。Block size 较大时，block 数量更少，因此最终执行的 atomicAdd 次数也更少，可以降低竞争和全局内存访问开销，从而提升 reduction 性能。

Exercise 4: Algorithm Analysis

Question 1: In tree reduction, what fraction of threads are idle after the first step?

Your answer:

在 tree reduction 的第一步之后，只有一半线程继续参与下一轮计算，另一半线程变为空闲。因此空闲线程比例是 1/2，也就是 50%。

Question 2: Why does warp shuffle not require `__syncthreads()` ?

Your answer:

warp shuffle 不需要 `__syncthreads()` ，因为它只在同一个 warp/wavefront 内部的线程之间交换数据。同一个 warp/wavefront 内的线程以 lockstep 方式执行，天然保持同步，所以不需要 block 级别的同步。

Question 3: How does multi-element per thread improve memory bandwidth utilization?

Your answer:

multi-element per thread 让每个线程在进入 shared memory tree reduction 之前，先从全局内存读取并累加多个元素。这样可以减少 block 数量和最终 atomic 操作次数，同时降低同步和 shared memory reduction 的相对开销。由于每个线程处理更多连续数据，GPU 可以更充分地利用全局内存带宽。

12. Challenge Exercises

Challenge 1: Implement Tree Reduction

Modify `main.hip` to use shared memory tree reduction instead of naive atomics.

`reduction<<<threadperblock, blockpergrid>>>(d_input, d_output, N);` 修改为 `reduction<<<blockpergrid, threadperblock, sharedMemSize>>>(d_input, d_output, N);`

Challenge 2: Add Warp Shuffle

For the last 64 threads, use warp shuffle intrinsics:

- `__shfl_down()` for AMD GPUs
- Eliminate `__syncthreads()` for intra-warp operations

```
__global__ void reduction(const int* input, int* output, int N) {
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (idx < N) ? input[idx] : 0;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride >= warpSize; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid < warpSize) {
        int val = sdata[tid];

        for (int offset = warpSize / 2; offset > 0; offset >>= 1) {
            val += __shfl_down(val, offset);
        }

        if (tid == 0) {
            atomicAdd(output, val);
        }
    }
}
```

Challenge 3: Multi-Pass Reduction

For very large arrays, implement a two-pass reduction:

1. First pass: Reduce to N/block_size partial sums
2. Second pass: Reduce partial sums to final result

```
__global__ void reduce_to_partials(const int* input, int* partials, int N) {
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (idx < N) ? input[idx] : 0;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride >= warpSize; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid < warpSize) {
        int val = sdata[tid];

        for (int offset = warpSize / 2; offset > 0; offset >>= 1) {
            val += __shfl_down(val, offset);
        }

        if (tid == 0) {
            partials[blockIdx.x] = val;
        }
    }
}
```

```
extern "C" void solve(const int* input, int* output, int N) {
    int *d_input, *d_partials_1, *d_partials_2;

    hipMalloc(&d_input, N * sizeof(int));
    hipMemcpy(d_input, input, N * sizeof(int), hipMemcpyHostToDevice);

    int threadperblock = 256;
    int sharedMemSize = threadperblock * sizeof(int);

    int max_partials = (N + threadperblock - 1) / threadperblock;
    hipMalloc(&d_partials_1, max_partials * sizeof(int));
    hipMalloc(&d_partials_2, max_partials * sizeof(int));

    int num_items = N;
    int* current_input = d_input;
    int* current_output = nullptr;
    bool use_first_buffer = true;

    while (num_items > 1) {
        int blocks = (num_items + threadperblock - 1) / threadperblock;
        current_output = use_first_buffer ? d_partials_1 : d_partials_2;

        reduce_to_partials<<<blocks, threadperblock, sharedMemSize>>>(
            current_input,
            current_output,
            num_items
        );
        hipDeviceSynchronize();

        current_input = current_output;
        num_items = blocks;
        use_first_buffer = !use_first_buffer;
    }

    hipMemcpy(output, current_input, sizeof(int), hipMemcpyDeviceToHost);

    hipFree(d_input);
    hipFree(d_partials_1);
    hipFree(d_partials_2);
}
```

Challenge 4: Benchmark Different Operations

Modify the kernel to compute:

- Maximum value
- Minimum value
- Product (be careful of overflow)

Compare performance differences.

```
enum ReduceOp {
    OP_SUM = 0,
    OP_MAX = 1,
    OP_MIN = 2,
    OP_PRODUCT = 3
};

__device__ int identity_value(int op) {
    if (op == OP_SUM) return 0;
    if (op == OP_MAX) return INT_MIN;
    if (op == OP_MIN) return INT_MAX;
    return 1;
}

__device__ int apply_op(int a, int b, int op) {
    if (op == OP_SUM) return a + b;
    if (op == OP_MAX) return max(a, b);
    if (op == OP_MIN) return min(a, b);
    return a * b;
}

__global__ void reduce_to_partials_op(const int* input, int* partials, int N, int op) {
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    int identity = identity_value(op);
    sdata[tid] = (idx < N) ? input[idx] : identity;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride >= warpSize; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] = apply_op(sdata[tid], sdata[tid + stride], op);
        }
        __syncthreads();
    }

    if (tid < warpSize) {
        int val = sdata[tid];

        for (int offset = warpSize / 2; offset > 0; offset >>= 1) {
            int other = __shfl_down(val, offset);
            val = apply_op(val, other, op);
        }

        if (tid == 0) {
            partials[blockIdx.x] = val;
        }
    }
}

const char* op_name(int op) {
    if (op == OP_SUM) return "Sum";
    if (op == OP_MAX) return "Max";
    if (op == OP_MIN) return "Min";
    return "Product";
```

```
}

struct BenchResult {
    float time_ms;
    int result;
};

BenchResult run_reduction_op(const int* d_input, int N, int op, int block_size) {
    int max_partials = (N + block_size - 1) / block_size;

    int *d_partials_1, *d_partials_2;
    hipMalloc(&d_partials_1, max_partials * sizeof(int));
    hipMalloc(&d_partials_2, max_partials * sizeof(int));

    int sharedMemSize = block_size * sizeof(int);

    hipEvent_t start, stop;
    hipEventCreate(&start);
    hipEventCreate(&stop);

    hipEventRecord(start);

    int num_items = N;
    const int* current_input = d_input;
    int* current_output = d_partials_1;

    while (num_items > 1) {
        int blocks = (num_items + block_size - 1) / block_size;

        if (current_input == d_partials_1) {
            current_output = d_partials_2;
        } else {
            current_output = d_partials_1;
        }

        reduce_to_partials_op<<<blocks, block_size, sharedMemSize>>>(
            current_input,
            current_output,
            num_items,
            op
        );

        current_input = current_output;
        num_items = blocks;
    }

    hipEventRecord(stop);
    hipEventSynchronize(stop);

    float ms = 0.0f;
    hipEventElapsedTime(&ms, start, stop);

    int h_result = 0;
    hipMemcpy(&h_result, current_input, sizeof(int), hipMemcpyDeviceToHost);

    hipEventDestroy(start);
    hipEventDestroy(stop);
    hipFree(d_partials_1);
```

```
    hipFree(d_partials_2);

    return {ms, h_result};
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        std::cerr << "Usage: " << argv[0] << " <input_file>" << std::endl;
        return 1;
    }

    std::ifstream fin(argv[1]);
    if (!fin.is_open()) {
        std::cerr << "Cannot open " << argv[1] << std::endl;
        return 1;
    }

    int N;
    fin >> N;

    std::vector<int> h_input(N);
    for (int i = 0; i < N; i++) {
        fin >> h_input[i];
    }
    fin.close();

    int* d_input;
    hipMalloc(&d_input, (size_t)N * sizeof(int));
    hipMemcpy(d_input, h_input.data(), (size_t)N * sizeof(int), hipMemcpyHostToDevice);

    hipDeviceProp_t prop;
    hipGetDeviceProperties(&prop, 0);

    int block_size = 256;

    std::cout << "=== Reduction Operation Benchmark ===" << std::endl;
    std::cout << "Device: " << prop.name << std::endl;
    std::cout << "N = " << N << ", Block size = " << block_size << std::endl;
    std::cout << std::endl;

    printf("%-12s %-12s %-15s\n", "Operation", "Time (ms)", "Result");
    printf("%-12s %-12s %-15s\n", "---------", "---------", "------");

    int ops[] = {OP_SUM, OP_MAX, OP_MIN, OP_PRODUCT};

    for (int op : ops) {
        BenchResult result = run_reduction_op(d_input, N, op, block_size);
        printf("%-12s %-12.4f %-15d\n", op_name(op), result.time_ms, result.result);
    }

    std::cout << std::endl;
    std::cout << "Note: Product may overflow for large integer inputs, so compare timing rather than result correctnes

    hipFree(d_input);
    return 0;
}
```

Operation	Time (ms)	Result
Sum	0.3074	-74993790
Max	0.0289	-50
Min	0.0305	-100
Product	0.0305	0

从结果可以看到，max、min 和 product 的执行时间比较接近，都在 0.03 ms 左右；sum 明显更慢，为 0.3074 ms。这里的 sum 结果需要累加大量整数，可能受到整数加法链、归约过程和数据分布的影响，而 max/min 只需要比较并保留一个值，product 在大规模整数输入下很容易溢出，最终结果为 0，因此更适合比较运行时间而不是结果正确性。
总体来说，不同 reduction operation 的性能会有差异。即使 kernel 结构相同，具体操作的计算代价、数据依赖、溢出行为以及编译器优化方式都会影响最终性能。

lab5

Exercise 1: Understand the Algorithm

Question 1: Why is the division by `n_samples` done in each thread instead of once at the end?

Your answer:

这里把除以 `n_samples` 放在每个线程中，是为了让每个线程直接计算自己对最终积分结果的贡献值：

```
tmp = (b - a) * y_samples[idx] / n_samples
```

这样所有线程通过 `atomicAdd` 累加到 `result` 后， `result` 本身就是最终积分近似值，不需要再额外做一次除法。不过从优化角度看，也可以先累加所有 `y_samples[idx]` ，最后只除以一次 `n_samples` ，这样通常会减少重复计算。

Question 2: What is the time complexity of the atomic reduction approach?

Your answer:

atomic reduction 方法的总体工作量是 O(N)，因为每个 sample 都需要被一个线程处理一次，并且每个线程都会执行一次 `atomicAdd` 。但是由于所有线程都竞争更新同一个 `result` ，atomic 操作会产生严重竞争，实际性能可能因为串行化而变差。

Question 3: How could you modify this kernel to use shared memory reduction instead of atomics?

Your answer:

可以让每个 block 先把线程计算出的局部结果存入 shared memory，然后在 block 内使用 tree reduction 将这些值归约成一个 partial sum。最后只让每个 block 的 thread 0 对全局 `result` 执行一次 `atomicAdd` 。这样 global atomic 的次数从每个线程一次减少到每个 block 一次，可以显著降低 atomic contention。

Exercise 2: Manual Calculation

For Test 1, verify the result manually:

Given: $a = 0, b = 2, n = 8$ samples

Formula: $\text{result} = \frac{(b-a)}{n} \sum_{i=1}^{n} y_i = \frac{2}{8} \times \sum y_i$

Your calculation:

- Sum of y values: `23.7507`
- Expected result: `5.937675`
- Does it match the program output? `Yes`

Exercise 3: Scalability Analysis

Record the execution times:

Test	Sample Count	Execution Time
4	100,000	0.097 s
7	10,000,000	0.968 s
8	100,000,000	8.430 s

Question 1: When sample count increases 100x (from test 4 to 7), how much does execution time increase?

Your answer:

从 test 4 到 test 7，sample count 从 100,000 增加到 10,000,000，增加了 100 倍。执行时间从 0.097 s 增加到 0.968 s，大约为10倍

Question 2: Is the scaling linear? Why or why not?

Your answer:
不是完全线性的。如果是线性 scaling，sample count 增加 100 倍，执行时间也应该接近增加 100 倍。但实际执行时间只增加了大约 10 倍。原因是 GPU 可以并行处理大量 sample，sample 数量增加后 GPU 利用率更高，因此运行时间不会严格按 sample 数量同比例增加。同时，总运行时间还包含文件读取、内存分配、Host 到 Device 数据传输、kernel 启动开销以及 atomic reduction 等开销，所以整体 scaling 不是简单的线性关系。

Exercise 4: Bottleneck Analysis

Question 1: If the computation takes 1 cycle and atomic takes 100 cycles, what is the theoretical efficiency for N=1M threads?

Hint: Total work = N x (compute + atomic)

Your calculation: efficiency = 1 / 101 ≈ 0.0099 = 0.99%

Question 2: How would shared memory reduction improve this?

Your answer:
shared memory reduction 可以让每个 block 先在 shared memory 中完成局部归约，然后只让每个 block 的一个线程对全局结果执行一次 atomicAdd。这样 global atomic 的次数从每个线程一次减少到每个 block 一次，大幅降低 atomic contention。由于 shared memory 访问比 global atomic 更快，整体性能会明显提升。

Question 3: With 256 threads per block and N=1M, how many global atomics would shared memory reduction require?

Your calculation: ceil(1,000,000 / 256) = 3,907

Exercise 5: Precision Considerations

Question 1: The kernel uses `double` (64-bit). How many significant digits does double precision provide?

Your answer:

`double` 是 64-bit 浮点数，通常提供大约 15 到 17 位十进制有效数字。

Question 2: If we switch to `float` (32-bit), what problems might occur with N=100M samples?

Your answer:

`float` 是 32-bit 浮点数，通常只有大约 6 到 7 位十进制有效数字。对于 N=100M 这样的大规模样本，累加大量 `float` 值时会产生更明显的舍入误差。很多较小的增量可能在累加到一个较大的总和时被丢失，导致最终积分结果精度下降。此外，parallel reduction 的累加顺序不同，也可能让误差更加不稳定。因此使用 `float` 可能会比 `double` 得到更不准确的结果。

Exercise 6: Optimization Analysis

Question 1: In the optimized version, how many global atomics occur for N=1M with 256 blocks?

Your answer:

在优化版本中，每个 block 只执行一次 global `atomicAdd`。如果使用 256 个 blocks，那么 global atomics 的数量就是：

```
256
```

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x)`?

Your answer:

grid-stride loop 的作用是让每个线程可以处理多个 sample，而不是只处理一个 sample。循环步长是整个 grid 的线程总数：

```
blockDim.x * gridDim.x
```

这样即使 `n_samples` 比总线程数大很多，所有 sample 仍然都能被处理。同时，这种写法可以减少需要启动的 block 数量，提高线程复用率，并让 kernel 能适应不同规模的输入数据。

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer:

因为每个 block 内已经先把多个 `y_samples` 累加成了一个 partial sum，所以只需要在最终写入全局结果时，对这个 partial sum 乘一次 `(b - a) / n_samples`。让 thread 0 执行这个操作可以避免每个线程重复做相同的乘除法，减少计算开销。同时，thread 0 也是负责把 block 的 partial result 通过 `atomicAdd` 写入全局结果的线程。

11. Challenge Exercises

Challenge 1: Implement Shared Memory Reduction

Modify `main.hip` to use the two-phase approach shown in Section 9. Compare performance.

```
__global__ void montecarlo(const double* y_samples, double* result,
                           double a, double b, int n_samples) {
    extern __shared__ double sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    double local_value = 0.0;
    if (idx < n_samples) {
        local_value = y_samples[idx];
    }

    sdata[tid] = local_value;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid == 0) {
        double scale = (b - a) / n_samples;
        atomicAdd(result, sdata[0] * scale);
    }
}

void solve(const double* y_samples, double* result, double a, double b, int n_samples) {
    double *d_ysamples, *d_result;

    hipMalloc(&d_ysamples, n_samples * sizeof(double));
    hipMalloc(&d_result, sizeof(double));

    hipMemcpy(d_ysamples, y_samples, n_samples * sizeof(double), hipMemcpyHostToDevice);
    hipMemset(d_result, 0, sizeof(double));

    int threadperblock = 256;
    int blockpergrid = (n_samples + threadperblock - 1) / threadperblock;
    int sharedMemSize = threadperblock * sizeof(double);

    montecarlo<<<blockpergrid, threadperblock, sharedMemSize>>>(
        d_ysamples,
        d_result,
        a,
        b,
        n_samples
    );
    hipDeviceSynchronize();

    hipMemcpy(result, d_result, sizeof(double), hipMemcpyDeviceToHost);

    hipFree(d_ysamples);
    hipFree(d_result);
}
```

- 测试结果：

Testcase	Output
testcases/1.in	5.937675
testcases/4.in	751909.082340

这些结果和原来的 atomic 实现一致，说明 shared memory reduction 版本是正确的。这个优化把 global atomic 操作次数从“每个线程一次”减少到“每个 block 一次”，因此可以降低 atomic contention，提高性能。

Challenge 2: Compute Pi Using Monte Carlo

Classic application: estimate π by sampling points in a square and counting how many fall inside a quarter circle.

$$\pi \approx 4 \times \frac{\text{points inside circle}}{\text{total points}}$$

```
__device__ unsigned int lcg_random(unsigned int seed) {
    return 1664525u * seed + 1013904223u;
}

__device__ double random_double(unsigned int value) {
    return static_cast<double>(value) / static_cast<double>(UINT_MAX);
}

__global__ void estimate_pi_kernel(int* block_counts, int n_samples) {
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    int inside = 0;

    if (idx < n_samples) {
        unsigned int seed = idx * 747796405u + 2891336453u;

        seed = lcg_random(seed);
        double x = random_double(seed);

        seed = lcg_random(seed);
        double y = random_double(seed);

        if (x * x + y * y <= 1.0) {
            inside = 1;
        }
    }

    sdata[tid] = inside;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid == 0) {
        block_counts[blockIdx.x] = sdata[0];
    }
}

void solve(int n_samples, double* result) {
    int threadperblock = 256;
    int blockpergrid = (n_samples + threadperblock - 1) / threadperblock;
    int sharedMemSize = threadperblock * sizeof(int);

    int* d_block_counts;
    hipMalloc(&d_block_counts, blockpergrid * sizeof(int));

    estimate_pi_kernel<<<blockpergrid, threadperblock, sharedMemSize>>>(
        d_block_counts,
        n_samples
    );
```

```
    hipDeviceSynchronize();

    std::vector<int> h_block_counts(blockpergrid);
    hipMemcpy(
        h_block_counts.data(),
        d_block_counts,
        blockpergrid * sizeof(int),
        hipMemcpyDeviceToHost
    );

    long long total_inside = 0;
    for (int count : h_block_counts) {
        total_inside += count;
    }

    *result = 4.0 * static_cast<double>(total_inside) / static_cast<double>(n_samples);

    hipFree(d_block_counts);
}
```

计算公式为：

```
pi ≈ 4 × inside_count / total_count
```

测试结果：

Sample Count	Estimated Pi
1,000,000	3.141824
10,000,000	3.141600

结果都非常接近真实的 π 值 `3.1415926...`。当 sample count 从 1,000,000 增加到 10,000,000 时，估计值从 `3.141824` 变为 `3.141600`，更接近真实值。这符合 Monte Carlo 方法的特点：样本数量越多，估计误差通常越小，误差大约按 `1 / sqrt(N)` 的速度下降。

Challenge 3: Error Analysis

For a known integral (e.g., $\int_0^1 x^2 dx = 1/3$):

1. Run with N = 1000, 10000, 100000, 1000000
2. Calculate absolute error for each
3. Verify the $1/\sqrt{n}$ convergence rate

```
__device__ unsigned int lcg_random(unsigned int seed) {
    return 1664525u * seed + 1013904223u;
}

__device__ double random_double(unsigned int value) {
    return static_cast<double>(value) / static_cast<double>(UINT_MAX);
}

__global__ void integrate_x2_kernel(double* block_sums, int n_samples) {
    extern __shared__ double sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    double value = 0.0;

    if (idx < n_samples) {
        unsigned int seed = idx * 747796405u + 2891336453u;
        seed = lcg_random(seed);

        double x = random_double(seed);
        value = x * x;
    }

    sdata[tid] = value;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid == 0) {
        block_sums[blockIdx.x] = sdata[0];
    }
}

double solve(int n_samples) {
    int threadperblock = 256;
    int blockpergrid = (n_samples + threadperblock - 1) / threadperblock;
    int sharedMemSize = threadperblock * sizeof(double);

    double* d_block_sums;
    hipMalloc(&d_block_sums, blockpergrid * sizeof(double));

    integrate_x2_kernel<<<blockpergrid, threadperblock, sharedMemSize>>>(
        d_block_sums,
        n_samples
    );
    hipDeviceSynchronize();

    std::vector<double> h_block_sums(blockpergrid);
    hipMemcpy(
        h_block_sums.data(),
        d_block_sums,
```

```
        blockpergrid * sizeof(double),
        hipMemcpyDeviceToHost
    );

    double total = 0.0;
    for (double partial : h_block_sums) {
        total += partial;
    }

    hipFree(d_block_sums);

    return total / static_cast<double>(n_samples);
}
```

我使用已知积分：

```
∫_0^1 x^2 dx = 1/3 ≈ 0.3333333333
```

分别测试了不同 sample count 下的 Monte Carlo 估计结果，并计算 absolute error。

N	Estimate	Exact	Absolute Error
1,000	0.3342499844	0.3333333333	0.0009166510
10,000	0.3333935295	0.3333333333	0.0000601962
100,000	0.3333371878	0.3333333333	0.0000038545
1,000,000	0.3333337343	0.3333333333	0.0000004009

从结果可以看到，随着 sample count 增加，估计值越来越接近真实值 `1/3`，absolute error 明显下降。

Monte Carlo 方法的理论误差通常满足：

```
Error ∝ 1 / sqrt(N)
```

也就是说，如果希望误差减少约 10 倍，通常需要将 sample count 增加约 100 倍。实验结果中，N 从 1,000 增加到 1,000,000，误差从 `0.0009166510` 降低到 `0.0000004009`，整体趋势符合 sample count 越大、误差越小的 Monte Carlo 收敛规律。

Challenge 4: Compare float vs double

Modify the kernel to use `float` instead of `double`. Test with large N and compare:

- Accuracy of result
- Execution time

```
__device__ unsigned int lcg_random(unsigned int seed) {
    return 1664525u * seed + 1013904223u;
}
__device__ double random_double(unsigned int value) {
    return static_cast<double>(value) / static_cast<double>(UINT_MAX);
}
__device__ float random_float(unsigned int value) {
    return static_cast<float>(value) / static_cast<float>(UINT_MAX);
}
__global__ void integrate_x2_double_kernel(double* block_sums, int n_samples) {
    extern __shared__ double sdata_double[];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    double value = 0.0;
    if (idx < n_samples) {
        unsigned int seed = idx * 747796405u + 2891336453u;
        seed = lcg_random(seed);

        double x = random_double(seed);
        value = x * x;
    }
    sdata_double[tid] = value;
    __syncthreads();
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata_double[tid] += sdata_double[tid + stride];
        }
        __syncthreads();
    }
    if (tid == 0) {
        block_sums[blockIdx.x] = sdata_double[0];
    }
}
__global__ void integrate_x2_float_kernel(float* block_sums, int n_samples) {
    extern __shared__ float sdata_float[];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float value = 0.0f;
    if (idx < n_samples) {
        unsigned int seed = idx * 747796405u + 2891336453u;
        seed = lcg_random(seed);
        float x = random_float(seed);
        value = x * x;
    }
    sdata_float[tid] = value;
    __syncthreads();
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata_float[tid] += sdata_float[tid + stride];
        }
        __syncthreads();
    }
    if (tid == 0) {
        block_sums[blockIdx.x] = sdata_float[0];
    }
}
double solve_double(int n_samples, float* elapsed_ms) {
```

```
    int threadperblock = 256;
    int blockpergrid = (n_samples + threadperblock - 1) / threadperblock;
    int sharedMemSize = threadperblock * sizeof(double);
    double* d_block_sums;
    hipMalloc(&d_block_sums, blockpergrid * sizeof(double));
    hipEvent_t start, stop;
    hipEventCreate(&start);
    hipEventCreate(&stop);
    hipEventRecord(start);
    integrate_x2_double_kernel<<<blockpergrid, threadperblock, sharedMemSize>>>(
        d_block_sums,
        n_samples
    );
    hipEventRecord(stop);
    hipEventSynchronize(stop);
    hipEventElapsedTime(elapsed_ms, start, stop);
    std::vector<double> h_block_sums(blockpergrid);
    hipMemcpy(
        h_block_sums.data(),
        d_block_sums,
        blockpergrid * sizeof(double),
        hipMemcpyDeviceToHost
    );
    double total = 0.0;
    for (double partial : h_block_sums) {
        total += partial;
    }
    hipEventDestroy(start);
    hipEventDestroy(stop);
    hipFree(d_block_sums);
    return total / static_cast<double>(n_samples);
}
float solve_float(int n_samples, float* elapsed_ms) {
    int threadperblock = 256;
    int blockpergrid = (n_samples + threadperblock - 1) / threadperblock;
    int sharedMemSize = threadperblock * sizeof(float);
    float* d_block_sums;
    hipMalloc(&d_block_sums, blockpergrid * sizeof(float));
    hipEvent_t start, stop;
    hipEventCreate(&start);
    hipEventCreate(&stop);
    hipEventRecord(start);
    integrate_x2_float_kernel<<<blockpergrid, threadperblock, sharedMemSize>>>(
        d_block_sums,
        n_samples
    );
    hipEventRecord(stop);
    hipEventSynchronize(stop);
    hipEventElapsedTime(elapsed_ms, start, stop);
    std::vector<float> h_block_sums(blockpergrid);
    hipMemcpy(
        h_block_sums.data(),
        d_block_sums,
        blockpergrid * sizeof(float),
        hipMemcpyDeviceToHost
    );
    float total = 0.0f;
    for (float partial : h_block_sums) {
```

```
        total += partial;
    }
    hipEventDestroy(start);
    hipEventDestroy(stop);
    hipFree(d_block_sums);
    return total / static_cast<float>(n_samples);
}
```

我修改了 kernel，分别使用 `double` 和 `float` 来计算已知积分：

```
∫_0^1 x^2 dx = 1/3 ≈ 0.3333333333
```

测试结果如下：

N	Double Estimate	Double Error	Double Time (ms)	Float Estimate	Float Error	Float Time (ms)
1,000,000	0.3333337343	0.0000004009	0.7689549923	0.3333333731	0.0000000397	0.0775040016
10,000,000	0.3333332294	0.0000001040	1.4720870256	0.3333337605	0.0000004272	0.3289340138
100,000,000	0.3333332505	0.0000000829	8.9454603195	0.3334860206	0.0001526872	1.6004899740

从执行时间看，`float` 明显比 `double` 更快。例如在 N = 100,000,000 时，double kernel time 为 8.9455 ms，而 float kernel time 为 1.6005 ms，float 大约快了 5.6 倍。

从精度看，小规模数据下 float 的误差也可能很小，但当 N 增大到 100,000,000 时，float 的误差明显变大，达到 0.0001526872，而 double 的误差仍然只有 0.0000000829。这是因为 float 只有大约 6 到 7 位十进制有效数字，累加大量样本时更容易产生舍入误差；double 有大约 15 到 17 位有效数字，因此在大规模累加时数值精度更稳定。

总体来说，`float` 速度更快，但在大规模 Monte Carlo 积分中可能损失明显精度；`double` 速度较慢，但结果更准确、更稳定。

lab6

Exercise 1: Algorithm Analysis

Question 1: In the assignment kernel, what is the time complexity per thread if k = 100?

Your answer:

在 assignment kernel 中，每个线程负责一个数据点，并且需要计算这个点到所有 k 个 centroid 的距离。因此每个线程的时间复杂度是：

```
O(k)
```

当 `k = 100` 时，每个线程需要计算 100 次距离，所以 per-thread work 是 `O(100)`，也就是常数意义上的 100 次距离计算。

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer:

recalculation kernel 先使用 shared memory atomics，是为了让每个 block 在本地先累加属于各个 cluster 的点坐标和数量。shared memory 的访问速度比 global memory 更快，而且竞争范围只在一个 block 内。之后每个 block 再把局部结果通过较少次数的 global atomics 写回全局内存。这样可以减少大量线程直接竞争 global memory 的 atomic 操作，从而降低 atomic contention，提高性能。

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer:

如果某个 centroid 没有任何点分配给它，那么这个 cluster 的 count 会是 0，无法用 `sum / count` 计算新的 centroid，否则会发生除以 0。通常做法是保持这个 centroid 不变，或者重新初始化它到某个随机点/较远的点上。这个实现中通常会在 `count > 0` 时才更新 centroid；如果 `count == 0`，则保留原来的 centroid。

Exercise 2: Performance Analysis

Record the execution time for test 4:

- Sample size: 50,000
- Clusters (k): 50
- Max iterations: 100
- Execution time: `0.114 s`

Question: What is the dominant computation in each iteration?

Your answer:

每一轮迭代中的主要计算是 assignment phase，也就是为每个数据点寻找最近的 centroid。每个线程处理一个点，并计算该点到所有 `k` 个 centroid 的距离。

对于 `N` 个点和 `k` 个 clusters，每轮需要：

```
N × k
```

次距离计算。

在这个测试中：

```
N = 50,000
k = 50
```

所以每轮大约需要：

```
50,000 × 50 = 2,500,000
```

次距离计算。因此 assignment kernel 通常是每次迭代中的主要计算开销。

Exercise 3: Compute Intensity

For N = 50,000 points and k = 50 clusters:

Assignment kernel:

- Distance calculations per point: `50`
- Total distance calculations: `2,500,000`

Update kernel:

- Shared memory atomics: `150,000`
- Global atomics (with 256 threads/block): `29,400`

Exercise 4: Memory Optimization

Question 1: The centroids are read by every thread. How could you optimize this access pattern?

Your answer:

centroids 会被大量线程反复读取，因此可以把 centroids 缓存在更快的存储层中。常见优化方法包括：

1. 将 centroids 加载到 shared memory 中，让同一个 block 内的线程复用这些 centroid 数据。
2. 如果 k 较小，也可以利用 constant memory，因为所有线程经常读取相同的 centroid，适合 broadcast 访问模式。
3. 使用 structure-of-arrays 的布局，例如 `centroid_x[]` 和 `centroid_y[]` 分开存储，提高内存访问连续性和 cache 命中率。

这样可以减少对 global memory 的重复读取，提高 assignment kernel 的性能。

Question 2: If k = 1000, what would be the impact on cache performance?

Your answer:

如果 `k = 1000`，每个线程都需要读取 1000 个 centroid，centroid 数据量会明显变大。这样可能导致 cache 容量不足，centroid 数据无法全部保留在 cache 中，从而增加 cache miss。每个线程反复从 global memory 读取 centroid，会显著增加内存带宽压力。

同时，assignment kernel 的计算量也会从 `O(N × k)` 中的 k 部分明显增加，每个点需要计算 1000 次距离。因此 k 很大时，cache performance 和整体性能都会下降，需要使用 shared memory tiling 或分批加载 centroids 等优化方法。

Exercise 5: Convergence Analysis

Question 1: What is the convergence threshold in this implementation?

Your answer:

这个实现中的收敛阈值是：

```
1.0e-8f
```

代码中通过比较 centroid 前后两次位置变化的平方距离来判断是否收敛：

```
dx * dx + dy * dy < 1.0e-8f
```

如果某个 centroid 的移动距离平方小于 `1.0e-8f`，就认为它已经收敛。

Question 2: Why is it important to check convergence rather than always running max_iterations?

Your answer:

检查 convergence 可以避免不必要的迭代。如果所有 centroids 在达到 `max_iterations` 之前已经基本不再移动，继续迭代只会浪费 GPU 计算资源和运行时间。提前停止可以提高效率，减少 kernel launch、内存访问和 distance calculation 的开销。

同时，不同数据集的收敛速度不同。有些数据可能很快收敛，有些数据需要更多迭代。因此使用 convergence check 比固定运行 `max_iterations` 更灵活。

Question 3: How does the double-buffering (swapping prev and new centroids) help?

Your answer:

double-buffering 使用两组 centroid buffer：一组保存上一轮的 centroids，另一组保存当前轮新计算出的 centroids。每轮迭代开始时交换 `prev` 和 `new` 指针，这样可以避免在同一个数组中同时读旧值和写新值。

这种方法有两个好处：

1. assignment kernel 可以稳定地读取上一轮的 centroid。
2. update kernel 可以把新 centroid 写入另一个 buffer，不会覆盖旧 centroid。

这样既方便比较新旧 centroid 的移动距离来判断 convergence，也避免了额外的大量数据复制。

11. Challenge Exercises

Challenge 1: Support Larger k

Modify the kernel to use dynamic shared memory allocation:

```
__global__ void recalculate_centroid_kernel(const float* data_x, const float* data_y, const int* labels,
                                            int* centroid_point_nums, float* new_centroids_x, float* new_centroids_y,
                                            int sample_size, int k) {
    extern __shared__ unsigned char shared_mem[];

    float* new_centroids_x_shared = reinterpret_cast<float*>(shared_mem);
    float* new_centroids_y_shared = new_centroids_x_shared + k;
    int* centroid_point_nums_shared = reinterpret_cast<int*>(new_centroids_y_shared + k);

    for (int i = threadIdx.x; i < k; i += blockDim.x) {
        new_centroids_x_shared[i] = 0.0f;
        new_centroids_y_shared[i] = 0.0f;
        centroid_point_nums_shared[i] = 0;
    }
    __syncthreads();

    int point_id = blockIdx.x * blockDim.x + threadIdx.x;

    if (point_id < sample_size) {
        int cid = labels[point_id];

        atomicAdd(&new_centroids_x_shared[cid], data_x[point_id]);
        atomicAdd(&new_centroids_y_shared[cid], data_y[point_id]);
        atomicAdd(&centroid_point_nums_shared[cid], 1);
    }

    __syncthreads();

    for (int i = threadIdx.x; i < k; i += blockDim.x) {
        atomicAdd(&new_centroids_x[i], new_centroids_x_shared[i]);
        atomicAdd(&new_centroids_y[i], new_centroids_y_shared[i]);
        atomicAdd(&centroid_point_nums[i], centroid_point_nums_shared[i]);
    }
}
```

Challenge 2: Add Distance Output

Modify the program to output the sum of squared distances (inertia) after clustering.

```
__global__ void compute_distance_kernel(const float* data_x, const float* data_y,
                                        const int* labels,
                                        const float* centroids_x, const float* centroids_y,
                                        float* distances,
                                        int sample_size) {
    int point_id = blockIdx.x * blockDim.x + threadIdx.x;

    if (point_id < sample_size) {
        int cid = labels[point_id];

        float dx = data_x[point_id] - centroids_x[cid];
        float dy = data_y[point_id] - centroids_y[cid];

        distances[point_id] = sqrtf(dx * dx + dy * dy);
    }
}
```

测试 testcases/4.in 的结果为：

```
Total distance: 5.38148e+06
Average distance: 107.63
```

Challenge 3: Extend to 3D

Add support for 3D point clustering by adding z coordinates.

```
__global__ void find_nearest_centroids_kernel(const float* data_x, const float* data_y, const float* data_z,
                                              int* labels,
                                              const float* centroids_x, const float* centroids_y, const float* centroi
                                              int sample_size, int k) {
    int point_id = blockIdx.x * blockDim.x + threadIdx.x;

    if (point_id < sample_size) {
        int nearest_centroid_index = 0;
        float min_distance_sq = INFINITY;

        for (int centroid_index = 0; centroid_index < k; ++centroid_index) {
            float dx = centroids_x[centroid_index] - data_x[point_id];
            float dy = centroids_y[centroid_index] - data_y[point_id];
            float dz = centroids_z[centroid_index] - data_z[point_id];

            float dist = dx * dx + dy * dy + dz * dz;

            if (dist < min_distance_sq) {
                min_distance_sq = dist;
                nearest_centroid_index = centroid_index;
            }
        }

        labels[point_id] = nearest_centroid_index;
    }
}
```

```
__global__ void recalculate_centroid_kernel(const float* data_x, const float* data_y, const float* data_z,
                                            const int* labels,
                                            int* centroid_point_nums,
                                            float* new_centroids_x, float* new_centroids_y, float* new_centroids_z,
                                            int sample_size, int k) {
    extern __shared__ unsigned char shared_mem[];

    float* new_centroids_x_shared = reinterpret_cast<float*>(shared_mem);
    float* new_centroids_y_shared = new_centroids_x_shared + k;
    float* new_centroids_z_shared = new_centroids_y_shared + k;
    int* centroid_point_nums_shared = reinterpret_cast<int*>(new_centroids_z_shared + k);

    for (int i = threadIdx.x; i < k; i += blockDim.x) {
        new_centroids_x_shared[i] = 0.0f;
        new_centroids_y_shared[i] = 0.0f;
        new_centroids_z_shared[i] = 0.0f;
        centroid_point_nums_shared[i] = 0;
    }
    __syncthreads();

    int point_id = blockIdx.x * blockDim.x + threadIdx.x;

    if (point_id < sample_size) {
        int cid = labels[point_id];

        atomicAdd(&new_centroids_x_shared[cid], data_x[point_id]);
        atomicAdd(&new_centroids_y_shared[cid], data_y[point_id]);
        atomicAdd(&new_centroids_z_shared[cid], data_z[point_id]);
        atomicAdd(&centroid_point_nums_shared[cid], 1);
    }

    __syncthreads();

    for (int i = threadIdx.x; i < k; i += blockDim.x) {
        atomicAdd(&new_centroids_x[i], new_centroids_x_shared[i]);
        atomicAdd(&new_centroids_y[i], new_centroids_y_shared[i]);
        atomicAdd(&new_centroids_z[i], new_centroids_z_shared[i]);
        atomicAdd(&centroid_point_nums[i], centroid_point_nums_shared[i]);
    }
}
```

```
__global__ void finalize_recalculation_and_compare_kernel(const int* centroid_point_nums,
                                                          float* new_centroids_x, float* new_centroids_y, float* new_c
                                                          const float* prev_centroids_x, const float* prev_centroids_y
                                                          const float* prev_centroids_z,
                                                          int k, int* num_converged_centroids) {
    int idx = threadIdx.x;

    if (idx == 0) {
        *num_converged_centroids = 0;
    }
    __syncthreads();

    if (idx < k) {
        int count = centroid_point_nums[idx];

        if (count > 0) {
            new_centroids_x[idx] /= count;
            new_centroids_y[idx] /= count;
            new_centroids_z[idx] /= count;
        } else {
            new_centroids_x[idx] = prev_centroids_x[idx];
            new_centroids_y[idx] = prev_centroids_y[idx];
            new_centroids_z[idx] = prev_centroids_z[idx];
        }

        float dx = new_centroids_x[idx] - prev_centroids_x[idx];
        float dy = new_centroids_y[idx] - prev_centroids_y[idx];
        float dz = new_centroids_z[idx] - prev_centroids_z[idx];

        if (dx * dx + dy * dy + dz * dz < 1.0e-8f) {
            atomicAdd(num_converged_centroids, 1);
        }
    }
}
```

Challenge 4: K-Means++ Initialization

Implement K-Means++ initialization on GPU:

1. Choose first centroid randomly
2. For each subsequent centroid, choose with probability proportional to distance squared

```
void initialize_kmeans_plus_plus(const vector<float>& data_x,
                                 const vector<float>& data_y,
                                 const vector<float>& data_z,
                                 vector<float>& centroid_x,
                                 vector<float>& centroid_y,
                                 vector<float>& centroid_z,
                                 int sample_size,
                                 int k) {
    mt19937 rng(12345);

    int first = 0;
    centroid_x[0] = data_x[first];
    centroid_y[0] = data_y[first];
    centroid_z[0] = data_z[first];

    vector<float> min_dist_sq(sample_size, numeric_limits<float>::max());

    for (int c = 1; c < k; c++) {
        double total_dist = 0.0;

        for (int i = 0; i < sample_size; i++) {
            float dx = data_x[i] - centroid_x[c - 1];
            float dy = data_y[i] - centroid_y[c - 1];
            float dz = data_z[i] - centroid_z[c - 1];

            float dist = dx * dx + dy * dy + dz * dz;

            if (dist < min_dist_sq[i]) {
                min_dist_sq[i] = dist;
            }

            total_dist += min_dist_sq[i];
        }

        int selected = c % sample_size;

        if (total_dist > 0.0) {
            uniform_real_distribution<double> dist_gen(0.0, total_dist);
            double target = dist_gen(rng);
            double cumulative = 0.0;

            for (int i = 0; i < sample_size; i++) {
                cumulative += min_dist_sq[i];
                if (cumulative >= target) {
                    selected = i;
                    break;
                }
            }
        }

        centroid_x[c] = data_x[selected];
        centroid_y[c] = data_y[selected];
        centroid_z[c] = data_z[selected];
    }
}
```

K-Means++ 的基本思想是：

1. 先选择一个初始 centroid。
2. 对每个点，计算它到当前最近 centroid 的距离平方。
3. 按照距离平方作为权重选择下一个 centroid。
4. 重复直到选出 k 个 centroids。

这样可以让初始 centroids 更分散，通常比随机初始化更稳定，也更容易得到较好的聚类结果。

测试输入为 6 个 3D 点：

```
(0, 0, 0)
(1, 1, 1)
(2, 2, 2)
(10, 10, 10)
(11, 11, 11)
(12, 12, 12)
```

程序输出的 K-Means++ 初始 centroids 为：

```
(0, 0, 0)
(12, 12, 12)
```

最终 centroids 为：

```
(1, 1, 1)
(11, 11, 11)
```

这符合预期：前三个点被聚成一个 cluster，中心为 `(1, 1, 1)`；后三个点被聚成另一个 cluster，中心为 `(11, 11, 11)`。说明 K-Means++ 初始化和后续 3D K-Means 迭代都能正常工作。

Challenge 5: Mini-Batch K-Means

Process random subsets of points each iteration to speed up convergence on very large datasets.

```
__global__ void find_nearest_centroids_kernel(const float* data_x, const float* data_y, const float* data_z,
                                              int* labels,
                                              const float* centroids_x, const float* centroids_y, const float* centroi
                                              int sample_size, int k,
                                              int batch_start, int batch_size) {
    int local_id = blockIdx.x * blockDim.x + threadIdx.x;

    if (local_id < batch_size) {
        int point_id = (batch_start + local_id) % sample_size;

        int nearest_centroid_index = 0;
        float min_distance_sq = INFINITY;

        for (int centroid_index = 0; centroid_index < k; ++centroid_index) {
            float dx = centroids_x[centroid_index] - data_x[point_id];
            float dy = centroids_y[centroid_index] - data_y[point_id];
            float dz = centroids_z[centroid_index] - data_z[point_id];
            float dist = dx * dx + dy * dy + dz * dz;

            if (dist < min_distance_sq) {
                min_distance_sq = dist;
                nearest_centroid_index = centroid_index;
            }
        }

        labels[point_id] = nearest_centroid_index;
    }
}
```

```
__global__ void recalculate_centroid_kernel(const float* data_x, const float* data_y, const float* data_z,
                                            const int* labels,
                                            int* centroid_point_nums,
                                            float* new_centroids_x, float* new_centroids_y, float* new_centroids_z,
                                            int sample_size, int k,
                                            int batch_start, int batch_size) {
    extern __shared__ unsigned char shared_mem[];

    float* new_centroids_x_shared = reinterpret_cast<float*>(shared_mem);
    float* new_centroids_y_shared = new_centroids_x_shared + k;
    float* new_centroids_z_shared = new_centroids_y_shared + k;
    int* centroid_point_nums_shared = reinterpret_cast<int*>(new_centroids_z_shared + k);

    for (int i = threadIdx.x; i < k; i += blockDim.x) {
        new_centroids_x_shared[i] = 0.0f;
        new_centroids_y_shared[i] = 0.0f;
        new_centroids_z_shared[i] = 0.0f;
        centroid_point_nums_shared[i] = 0;
    }
    __syncthreads();

    int local_id = blockIdx.x * blockDim.x + threadIdx.x;

    if (local_id < batch_size) {
        int point_id = (batch_start + local_id) % sample_size;
        int cid = labels[point_id];

        atomicAdd(&new_centroids_x_shared[cid], data_x[point_id]);
        atomicAdd(&new_centroids_y_shared[cid], data_y[point_id]);
        atomicAdd(&new_centroids_z_shared[cid], data_z[point_id]);
        atomicAdd(&centroid_point_nums_shared[cid], 1);
    }

    __syncthreads();

    for (int i = threadIdx.x; i < k; i += blockDim.x) {
        atomicAdd(&new_centroids_x[i], new_centroids_x_shared[i]);
        atomicAdd(&new_centroids_y[i], new_centroids_y_shared[i]);
        atomicAdd(&new_centroids_z[i], new_centroids_z_shared[i]);
        atomicAdd(&centroid_point_nums[i], centroid_point_nums_shared[i]);
    }
}
```

测试输入为：

```
6 2 20 3
0 1 2 10 11 12
0 1 2 10 11 12
0 1 2 10 11 12
```

其中：

```
sample_size = 6
k = 2
max_iter = 20
batch_size = 3
```

K-Means++ 初始化结果为：

```
(0, 0, 0)
(12, 12, 12)
```

最终 centroids 输出为：

```
1 11
1 11
1 11
```

也就是最终 3D centroids 为：

```
(1, 1, 1)
(11, 11, 11)
```

结果符合预期，说明 mini-batch 版本可以正常工作。Mini-Batch K-Means 的优点是每轮迭代处理的数据更少，因此单轮计算更快；缺点是每轮只使用部分数据，更新方向可能更 noisy，通常需要更多迭代才能达到和 full-batch K-Means 接近的结果。